Specification and Simulation of Embedded Systems notes

Luca Seggiani
2026

1 Lesson 22-09-26

1.1 Introduction

The course aims to give an introduction to the use of **HDL**s (*Hardware Description Languages*):

- We will see HDLs used as *specification languages* used to design hardware. Both *Verilog* and *VHDL* are used, respectively industry and academic based;
- High level concepts will be studied at the **RT** (*Register Transfer*) level of abstraction. We'll also see design methodologies for **ASIC**s (*Application-Specific Integrated Circuits*) and **FPGA**s (*Field Programmable Gate Arrays*);
- Simulation tools are the main tool available for verification and validation of designs. Other important tools include **CAD**-based formal verification, and mathematical *formal verification* methods.

The topics covered will be:

- Usage of **simulation** tools such as the Xilinx Vivado suite of programs;
- **Combinatorial** networks, that is networks where the output value is only determined by the values taken by their inputs;
- **Sequential** networks, that is networks where the output value is determined both by the values taken by the inputs and internal state. Sequential networns can either be *asynchronous* or *synchronized* to an external clock signal;
- **FSM**s (*Finite State Machines*), which we will see as a specific class of sequential circuits;
- **FSM-D** models (FSMs operating at logical level, controlling *datapaths* for data manipulation);
- **RAM**s (*Random Access Memories*) and **ROM**s (*Read-Only Memories*), and their respective interfaces facing other components (including ones we will design ourselves).

1.1.1 VHDL as a specification language

VHDL will be used and explained in a design-oriented approach covering:

1. **Combinatorial** logic;
2. *Synchronous* **sequential** logic (with *asynchronous* sequential logic being mostly a less useful superset);
3. **Datapath** components.

Verilog will be covered, by *difference* from VHDL. Still, both languages are useful in academic and industrial contexts, and proficiency in both is expected in most design environments.

1.1.2 Register Transfer level designs

We will then move upwards to the **RT** (*Register Transfer*) level of abstraction. RT level is a design abstraction which models a synchronous digital circuit in terms of the flow of

digital signals (data) between hardware registers, and the logical operations performed on those signals.

Using this abstraction we will design embedded systems, covering design *metrics* (related to economical, manifacturing, ... factors), design *methodologies* for **single-purpose** processors.

By *single-purpose*, we mean processors that implement a specific algorithm in hardware (ASICs, FPGAs): this is the opposite concept to *general-purpose* computing.

FSM (*Finite State Machine*) optimization will also be covered.

1.1.3 Formal verification

The main tool used to verify a design is **simulation**, and simulation through the Xilinx suite of programs will be studied.

Another method that will be studied is **formal verification**, that is mathematical methods used to prove the validity of designs indipendent of the given inputs:

- *Theorem proving* involves expressing the hardware and desired property mathematically, then proving the property from axioms/rules;
- *Equivalence checking* entails, given two designs, proving that they behave equivalently under specified conditions;
- *Model checking* means determining if certain *properties* of our designed system can be proved to be true indipendently of the inputs.

1.2 VHDL for digital design

A **digital system** is an electronic system operating on data encoded as a sequence of zeroes and ones (low and high, off or on values).

When digital design started (aruond the 1960s), the number of components in a circuit was very limited. As of today, that number as grown up to billions of components on a single chip.

This growth has made the *documentation* process harder and harder, and the resulting artifacts more and more imprecise and ambiguous.

1.2.1 HDLs

The main problem is that *natural language* is not fit for description of a digital system. For this reason, machine-readable textual languages were invented for describing hardware. We call these languages **HDL**s (*Hardware Description Languages*).

HDLs are the opposite of *programming* languages: they don't describe *how* an algorithm is implemented, can't be executed, but describe the *temporal behavoir* and *spatial structure* of an hardware system. This allows to combine design and documentation in a single source of truth (documentation of a system can be extracted from its HDL description).

An HDL usually allows us to define:

- A **black box** representation of a given system, which only entails its inputs and outputs (which we call the *interface* of the system);
- An **architectural** view of the system, which entails how the inputs (and optionally, previous state) are combined to generate the outputs. The architecture of a system can be defined in terms of other systems (such as *And*, *Or* and *Not* gates), themselves taken as other black boxes with their own architectural implementation.

Of course, the set of components used to make up the architectural description of a system represents its *spatial structure*. The *temporal behavoir*, on the other hand, is specified using other language constructs that specify the timing constants between components.

1.2.2 VHDL

VHDL (*VHSIC Hardware Description Language*), with **VHSIC** (*Very High Speed Integrated Circuit*) being a project of the U.S. Dept. of Defense, is a hardware specification language based on a top-bottom approach. The syntax is similar to *Ada*, another U.S. Dept. of Defense creation. Today it's mainly used in academia, and is better suited to high-level behavioral modeling.

1.2.3 Verilog

Verilog was defined in 1995 at *Gateway Design Automation Inc.*, then acquired by *Cadence Design Systems*. The initial implementation was proprietary, and meant for designers to quickly iterate over prototypes in a bottom-up approach. Afterwards, the language was extended and made into an open standard. The syntax is C-like, and today it's the most significant difference from VHDL. Another difference is that Verilog is more fit for low-level work than VHDL.

1.2.4 Comparison of VHDL and Verilog

We said that Verilog will be studied by difference.

Other differences to note are in the table:

VHDL	Verilog
Ada-like, not *case sensitive*	C-like, *case sensitive*
Strongly typed, compile time error checking	Weakly typed, less compile time checking, more likely runtime bugs
Focused on safety	Focused on writing models quickly
User-defined types, 9 value logic	No user-defined types, 4 value logic
Hierarchy through `entity` definitions and implementation through `architecture`	Hierarchy through `module` definitions with self contained implementation
Gate level modeling with **FPGA** library support	Better **ASIC** support, user defined primitives
Assertions	No assertions
No named events	Named events
Partial interface abstraction via `component` definitions	No interface abstraction
Packaging via actual packages	Packaging via include files
Useful for high-level modeling	Better suited for low-level modeling
More verbose, meant to act both as design and documentation	Less verbose, no packaging capability, harder to extend
Unambiguous semantics and easily portable desings, no race conditions	Ambiguous emulation semantics, race conditions possible, different vendors might have different semantics
No simulation control defined in language	Basic simulation control defined in language

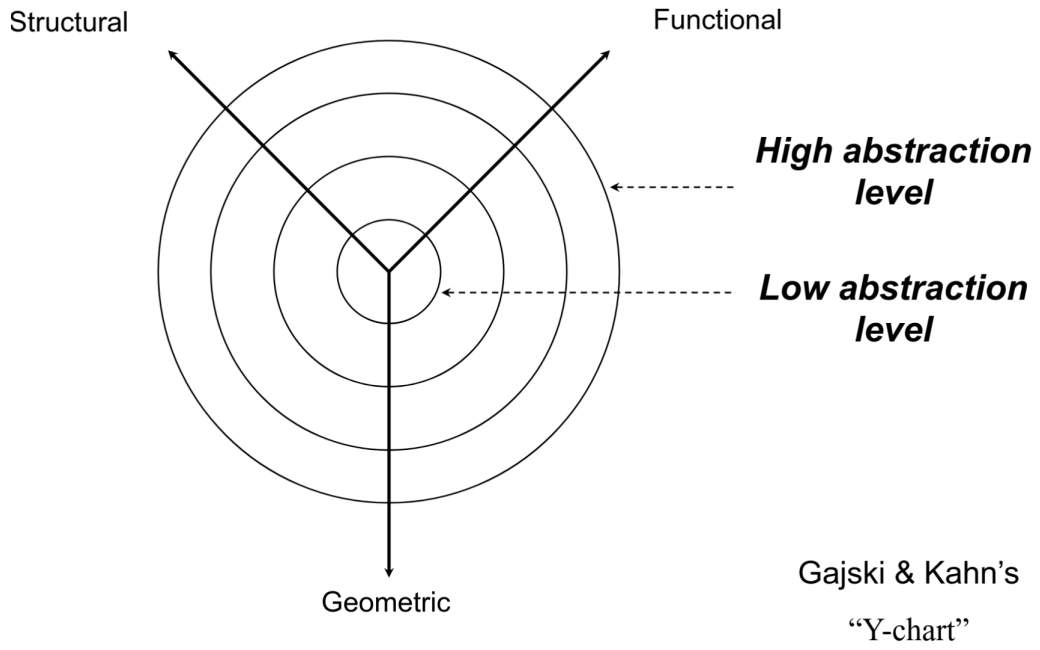
1.2.5 Synthesis

Today, HDLs are increasingly used for designing ICs in a *top-down* approach involving **synthesis**:

1. We *design* the circuit through high-level descriptions;
2. We leave the conversion to a low-level representation to *synthesis* tools, which automatically convert HDL behavior to HDL circuits;
3. The last step is, of course, *simulation* to validate the design against the original high-level description.

Synthesis is not the objective of this course. On the other hand, the use of HDLs for synthesis is growing as circuits get more complex, and synthesis tools are maturing. One thing to note is that HDLs were originally defined for simulation: some features of the language are not supported by synthesizers (for instance, timing constants don't necessarily translate to low-level components, neither do branching loops such as the `while` loop offered by HDLs).

1.2.6 The Y-chart



There are different ways of modeling a given system, each with different levels of abstraction:

- **Functional**: what the system *does*: this goes all the way from algorithmic (behavioral) representation to RT languages, boolean functions, up to the *differential equations* which govern the electronic components that make up the system;
- **Structural**: what the system is *made up* of: we can start from high-level blocks such as *processors* and *memories*, look at components which operate at RT level, up to single logic gates and their implementations based on transistors;
- **Geometric**: how the components of the system are *laid out* physically. This won't be of much interest in this course, as the focus is on *logical* modeling.

This relates to a set of ways we can model our systems:

- At the *system* level, we see systems as a set of interconnected high-level components and their interfaces (black boxes);
- At an *algorithmic* level, we see systems through the algorithms they implement;
- At the *RT* level, we see systems through registers and sequential networks that propagate values across them;
- At the *logic* level, we see systems through combinatorial networks (made up of gates) implementing boolean functions;
- At the *transistor* level, we see how gates themselves are implemented through transistors;
- At the *geometry/topology* level, we see how physical components (transistors) are laid out on boards.

The data type we use today in VHDL modeling is the `std_logic` type, which implements 9-value logic (compared to other versions, which use simple 2-value logic `bit` values).

2 Lesson 25-09-26

2.1 Introduction to VHDL

Let's start delving into the **VHDL** (*VHSIC Hardware Description Language*) language. There are 3 description styles in VHDL:

- Behavioral:
  - Specifies only the behavior at a higher level of abstraction.
  - Does not imply any particular structure or technology.
- Structural:
  - Specifies more details.
  - Components used and the structure of the interconnection between the components are clearly specified.
  - At a low level of abstraction
- Dataflow (*Register Transfer Language* (**RTL**)):
  - Data path and control signals are specified.
  - System is described in terms of the data transfer between registers.
  - At an intermediate level of abstraction

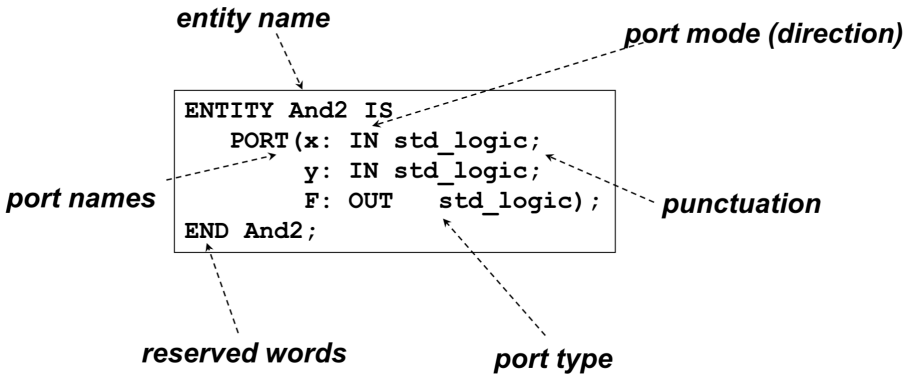
We also note that VHDL's syntax is Ada-based, case insensitive.

2.1.1 VHDL entities

A simple VHDL file contains a **design entity**, comprised of:

- An *entity declaration*;
- An *architecture body*.

Let's look at a very simple entity declaration:



As we know the declaration gives a *black box* view of the component, without its internal architecture.

`PORT` specifies a list of inputs and outputs of entity.

- `x: IN std_logic`: port `x` is an input port of type `std_logic`;
- `std_logic`: short for "standard logic", a logical bit type (versus say an integer type), which can be '0' or '1' (more later);
- Ports in list separated by semicolons.

A number of port modes are available:

- IN: Used for a signal that is an input to an entity.
- OUT: Used for a signal that is an output from an entity. The value of the signal can not be used inside the entity. This means that in an assignment statement, the signal can appear only to the left of the <= operator.
- INOUT: Used for a signal that is both an input to an entity and an output from the entity.
- BUFFER: Used for a signal that is an output from an entity. The value of the signal can be used inside the entity, which means that in an assignment statement, the signal can appear both on the left and right sides of the <= operator.

BUFFER differs from INOUT because of the following:

- It can't have more than 1 source;
- The only signal connected to it is another BUFFER port or signal with at most 1 source.

Logic types: VHDL supports different data types for logic. We'll mainly use **standard logic** as defined by IEEE 1164 standard (end of the 1980s).

- Values for simulation: 'U' (unaffected), 'X' (unknown), 'Z' (high impedance), 'W' (weak unknown), 'L' (weak 0), 'H' (weak 1), '-' (don't care), 0, 1
- package `std_logic_1164`: it contains the enumerated types `std_logic` and `std_ulogic`. The main difference between `std_logic` (standard logic) and `std_ulogic` (standard unresolved logic) is:
  - `u` stands for *unresolved* signal: it is illegal to force two values at the same time on a signal of type `std_ulogic`;
  - `std_logic`: there exist resolution functions that define what happens when two values are forced at the same time on a signal.

2.1.2 VHDL architectures

An **architecture** defines an entity's inside — an entity's functionality.

- It describes the behavior or the inner implementation of an entity;
- There may be several architectures for the same entity (for simulation, but not for synthesis);
- Each architecture must be bound to an entity.

One way to describe architecture is thorugh `PROCESS`: a `PROCESS` takes in arguments, and reacts to changes of those arguments by producing an output value. By *reactive*, we mean that the process is idle until something happens to the arguments, at which points it *reacts* by *doing something*. Because of this, the argument list is usually called the *sensitivity list*.

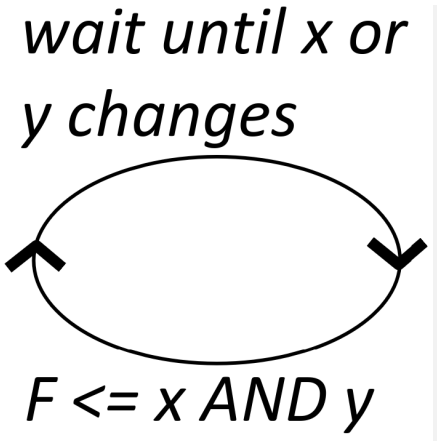
Let's take, for instance:

```
ENTITY And2 IS
    PORT (x: IN std_logic;
    y: IN std_logic;
    F: OUT std_logic);
END And2;

ARCHITECTURE And2_beh OF And2 IS
BEGIN

    PROCESS(x, y) -- this is the process
    BEGIN
        F <= x AND y;
    END PROCESS;

END And2_beh;
```



If `x` and `y` change in the above example, the process sets the output F to the logical AND of `x` and `y`. After executing its statements, the process waits until any of is arguments changes again, and then re-executes: a process is thus an infinite loop.

Concurrent statements: something we should go through is **concurrent** statements. We know that a typical VHDL architecture is:

```
ARCHITECTURE architecture_name OF name_of_entity IS
  -- Declarations
  -- components declarations
  -- signal declarations
  -- constant declarations
  -- function declarations
  -- procedure declarations
  -- type declarations

  BEGIN
    -- concurrent statements
END architecture_name;
```

The important part of an architecture are its *concurrent statements*. These are statements which can potentially execute at the same time, while interacting. There are a number of concurrent statements in VHDL:

- Concurrent *processes*;
- Concurrent signal assignments;
- Component instantiations;

- Procedure calls;
- Generate statements: these are particularly useful for procedural generation of components, which in hardware are often repeated. This explots the intrinsic *regularity* of hardware.

We focus on processes.

Processes model hardware's intrinsic concurrency:

- A process is one single concurrent operation;
- Statements within a process are sequential;
- *Concurrency* comes to mean multiple interacting processes.

Arocess declarations entails signals, variables and costants used in the process. These are to be enclosed between begin and end keywords. The statements are then sequentially executed.

A process is normally idle:

- If there is a sensitivity list (list of signals to which the process is sensitive), an event on one of the signals triggers the process
- If there is no sensitivity list, the process will always be in execution until a wait statement is reached
- Processes may alternatively have a sensitivity list or one or more wait statements (but not both)

Assignment to signals is carried out as follows:

- An assignment to a signal generates a projected value (projected value: value the signal will take later on);
- All signals in a process are all updated at the end of the execution of the process with their projected values;
- In the meantime, if the signal is used for a computation or assignment, the old value is used.

So we have *synchronization* at process level.

Important implications of this are that:

- Process execution order in a cycle doesn't matter;
- The order of assignments to different signals in a process doesn't matter.

One thing to note is that later signal assignment in process has precedence over earlier writ.

Simulation cycle: it can be useful to look at the simulation cycle, that is to consider how an HDL simulator actually works. HDL simulation is complex, so we'll introduce a simplified form. A simulator's job is to *determine values* for signals over time. It does so by repeatedly executing and suspending processes.

Let's look at the example:

```
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY SimEx1 IS
5   PORT (Q: OUT std_logic );
6 END SimEx1;
7
8 ARCHITECTURE SimEx1Arch OF SimEx1 IS
9   SIGNAL Clk, S: std_logic;
```

```
10
11  BEGIN
12  P1: PROCESS     -- clock generator
13    BEGIN
14    Clk <= '0';
15    WAIT FOR 10 NS;
16    Clk <= '1';
17    WAIT FOR 10 NS;
18  END PROCESS P1;
19
20  P2: PROCESS(S) -- output stage
21  BEGIN
22    Q <= NOT(S);
23  END PROCESS P2;
24
25  P3: PROCESS     -- clock divider
26  BEGIN
27    -- legacy syntax for rising edges
28    WAIT UNTIL Clk='1' AND Clk'EVENT;
29    S <= '1';
30    WAIT UNTIL Clk='1' AND Clk'EVENT;
31    S <= '0';
32    WAIT;
33  END PROCESS P3;
34
35 END SimEx1Arch;
```

Time (ns):	Init	0	10	10	10	20	30	30	30	40	50
Signals Q	'U'	'U'	'U'	'U'	*'0'*	'0'	'0'	'0'	*'1'*	'1'	'1'
Clk	'U'	*'0'*	*'1'*	'1'	'1'	*'0'*	*'1'*	'1'	'1'	*'0'*	*'1'*
S	'U'	'U'	'U'	*'1'*	'1'	'1'	'1'	*'0'*	'0'	'0'	'0'

Concurrent signal assignments can be seen as a sugared version of processes with a single signal assignment. The sensitivity list is automatically computed from the assignments right hand side arguments. They are used in dataflow descriptions for:

- Descriptions of combinational logic;
- Interconnection of components at a lower hierarchical level.

For instance, let's look at the example:

```
1  ARCHITECTURE Datafl OF fulladder IS
2  BEGIN
3    s  <= (a XOR b) XOR ci;
4    co <= (b AND ci) OR (a AND ci) OR (a AND b);
5  END Datafl;
6
7  -- translates to:
8
9  ARCHITECTURE Behav OF fulladder IS
10 BEGIN
11 PROCESS(a, b, ci)
12 BEGIN
13   s  <= (a XOR b) XOR ci;
14   co <= (b AND c) OR (a AND c) OR (a AND b);
15 END PROCESS;
16 END Behav;
```

2.1.3 VHDL testbenches

Once we define an entity, we want to study how our new entity behaves. We could do this manually, but we have software tools exactly for this reason. We perform a **simulation**: User provides input values, simulator generates output values.

- A simulation takes in **test vectors**, that is a sequence of input values;
- The result is a **waveform**, a graphical depiction of the sequence of values the simulation emits.

To do this we create new **testbench** entity that provides test vectors to component's inputs. An HDL testbench is:

- An entity with no ports;
- Where we deeclare component to test;
- Declare signal for each port;
- Instantiate component, map signals to ports;
- Process to set signal values at desired times.

An example of a testbench can be:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY Testbench IS
END Testbench;

ARCHITECTURE TBarch OF Testbench IS

    COMPONENT And2 IS
        PORT (
            x : IN  std_logic;
            y : IN  std_logic;
            F : OUT std_logic
        );
    END COMPONENT;

    SIGNAL x_s, y_s, F_s : std_logic;

BEGIN

    CompToTest : And2
        PORT MAP (x_s, y_s, F_s);

    PROCESS
    BEGIN
        -- Test all possible input combinations
        y_s <= '0';
        x_s <= '0';
        WAIT FOR 10 ns;

        y_s <= '0';
        x_s <= '1';
        WAIT FOR 10 ns;

        y_s <= '1';
        x_s <= '0';
        WAIT FOR 10 ns;

        y_s <= '1';
```

```
        x_s <= '1';
        WAIT;
    END PROCESS;

END TBarch;
```

2.2 Combinatorial circuits

A **circuit**, in the broadest sense, is connection of components. It's also known as a *structure*. A circuit is a second way to describe an architecture, in contrast to using a process, as earlier.

When used in a circuit, an entity is known as a `COMPONENT`. An *instance* is an occurrence of a component in a circuit. There may be multiple instances of a component.

This leads us to the third type of concurrent statements: **component instantiations**. Let's look at an example:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY BeltWarn IS -- root entity
    PORT (
        k : IN  std_logic;
        p : IN  std_logic;
        s : IN  std_logic;
        w : OUT std_logic
    );
END BeltWarn;

ARCHITECTURE Circuit OF BeltWarn IS -- root structural description

    COMPONENT And2 IS
        PORT (
            x : IN  std_logic;
            y : IN  std_logic;
            F : OUT std_logic
        );
    END COMPONENT;

    COMPONENT Inv IS
        PORT (
            x : IN  std_logic;
            F : OUT std_logic
        );
    END COMPONENT;

    SIGNAL n1, n2 : std_logic; -- signal declaration

BEGIN -- behavoir starts here

    And2_1 : And2 PORT MAP (k, p, n1);
    Inv_1  : Inv  PORT MAP (s, n2);
    And2_2 : And2 PORT MAP (n1, n2, w);

END Circuit;
```

When we create a circuit we have to:

- Declare a new entity;
- Declare its components.

We then want to connect components together. **Signal** declaration statements can be useful for this. A `SIGNAL` is like a variable in a traditional programming languages: it stores value, but at particular time. Signals may be explicitly declared, but an entity's ports are also signals.

We use signals in *port maps*.

Port maps are parts of *component instantiations* that map the ports of a component to other ports of the root entity, or signals defined within that entity.

A number of other methods of synthesizing combinatorial circuits exist. These are mainly based around finding the minimum set of components (AND, OR, NOT) needed to implement a certain boolean function. Mathematically, we say that this process involves finding a **SoP** (*Sum of Products*) form, where sums are to be taken as OR operations and products as AND operations. The SoP can be found by many methods, namely the *Quine McClusky* algorithm and the *Karnaugh Map* method. Some other help about this topic can be found in the author's notes at `https://seggiani-luca.github.io/appunti/triennale/2/reti-logiche.html`.

At this point some more things about *testbenches* should be clear:

- Note use of component instantiation statements;
- Signals are used because process cannot access instantiated component's ports directly;
- Component instantiation statements and process statements are concurrent statements and they can both appear in an architecture.

Hierarchycal design: component instantiation allows for designing entities in an hierarchical manner. This, alongside generation statements (which we introduced in 2.1.2, and will go in depth afterwards), is going to be one of our most powerful tools for logical design.

To summarize, an entity can be used as component in a new entity. This process can continue continue: a composed entity can be used as component in another entity, and so on. This makes hierarchy a powerful mechanism for managing complexity.

2.3 Top-down design

Let's introduce the **top-down** design philosophy. Designer may initially know system behavior, but not structure.

The idea of top-down design is to:

- Capture behavior, and simulate;
- Capture structure (circuit), simulate again;
- Gets behavior right first, unfettered by complexity of creating structure.

How can we describe behavior? One way in VHDL is to use a *process*. By making a process sensitive to its inputs, the process executes if a change occurs on any of those inputs. The simplest process uses one signal assignment statement.

We can then simulate the behavior captured by our process using testbench (same as shown earlier) to get waveforms. We can, this way, proceed to capture structure, simulate again using same testbench. The result should of course be the same waveforms.

It is useful to have signal assignment statements, as they assign values to signals which can be any combination of different operators. Note: A port is also a signal. Lastly, a process may have multiple signal assignment statements.

We will now cover a few more constructs which can be useful to capture high level behavior.

If statements are supported by VHDL. The condition *has* to be boolean, and errors will be reported at compile time.

We see an example:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY BeltWarn IS
  PORT (
    k, p, s : IN std_logic;
    w : OUT std_logic
  );
END BeltWarn;
ARCHITECTURE Behav OF BeltWarn IS
BEGIN
  PROCESS (k, p, s)
  BEGIN
    IF ((k AND p AND NOT(s)) = '1') THEN
      w <= '1';
    ELSE
      w <= '0';
    END IF;
  END PROCESS;
END Behav;
```

`ELSIF` is also supported, in Python fashion.

Vectors are provided by `std_logic_vector`. These represent collection of bits, which is more convenient than declaring separate bits like `d3, d2, d1, d0`. A nice feature is that a vector's bits are numbered: the ordering is specified in the syntax: `(0 TO 3), (1 TO 4)`, ... Assignment is possible with strings of bits.

An example is:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY Dcd2x4vect IS
  PORT (i: IN std_logic_vector(0 TO 1);
       d: OUT std_logic_vector(0 TO 3);
END Dcd2x4vect
```

Switch statements are also available, and they are called `CASE` statements. These are preferred over `IF-THEN-ELSE` when a single expression determines which statement(s) to execute. The selection keyword is `WHEN`, and the sequence of statements after a `WHEN` includes all statements between the `WHEN` itself and the next `WHEN` or `END CASE`.

An example is:

```
ARCHITECTURE Dcd2x4_vect OF Dcd2x4vect IS
BEGIN
  PROCESS (i)
  BEGIN
    CASE i IS
      WHEN "00" => d <= "0001";
      WHEN "01" => d <= "0010";
      WHEN "10" => d <= "0100";
      WHEN OTHERS => d <= "1000";
    END CASE;
  END PROCESS;
END Dcd2x4_vect
```

The *case sequential* statement is allowed, that is a `WHEN` clause can include more than one choice:

```
CASE expression IS
  WHEN choices =>
    -- sequential statements
  WHEN choices =>
    -- sequential statements
    -- (branches are allowed)
  WHEN OTHERS =>
    -- sequential statements
END CASE;
```

2.4 Design units

VHDL allows for **design units**. They are VHDL code segments that can be compiled separately and stored in a library for future use. Libraries are collections of compiled design units that are mapped to physical directories.A library is a set of components (entities and architectures) and packages.

There are 5 types of design units:

1. Entity;
2. Architecture;
3. Package;
4. Package body;
5. Configuration.

2.4.1 Packages

Packages are repositories for type definitions, procedures and functions. They group common declarations used by several design units. Standard packages are defined in generic libraries (e.g. IEEE) for numeric/text handling (for example, `std_logic_1164`, `numeric_std`, `std_logic_arith`, `std_logic_signed`, `std_logic_unsigned`, TEXTIO, ...).

A package consists of:

- A package **declaration** (with declarations of types, subtypes, constants, global signals, functions and procedures, attributes, files, components, ...).

  Because a package serves as a central repository for frequently used utilities, such as component declarations, ... the declarations may then be reused by any VHDL model by simply accessing the package.

  Usually, to use a package we use the `USE` clause.

  An example of a package declaration is:

```
PACKAGE example IS
  TYPE winter IS (Dec, Jan, Feb);
  COMPONENT And3 IS
    PORT (
      x : IN std_logic;
      y : IN std_logic;
      z : IN std_logic;
      F : OUT std_logic
    );
  END COMPONENT;
  CONSTANT pin2pin_delay : TIME := 125ns;
  FUNCTION int2bit_vect (int_val : INTEGER) RETURN bit_vector;
```

```
13 END example;
```

- A package **body**: it contains the implementations of functions, procedures, ... A package body is always associated with a package declaration.

  An example of a package body is:

```
1 PACKAGE BODY example IS
2   FUNCTION int2bit_vect (int_val : INTEGER) RETURN bit_vector IS
3     BEGIN
4       -- behavior of function described here
5     END int2bit_vect;
6 END example;
```

2.4.2 Configurations

VHDL description may consist of many design entities, each with several architectures, and organized into a design hierarchy. A **configuration** does the job of specifying the exact set of entities and architectures used in a particular simulation or synthesis run.

A configuration does two things:

- It specifies the design entity used in place of each component instance (i.e., it plugs the chip into the chip socket and then the socket-chip assembly into the PCB);
- It specifies the architecture to be used for each design entity (i.e., which die).

A configuration statement is used to bind a component instance to an entity-architecture pair. A configuration can be considered as a parts list for a design. It describes which behavior to use for each entity, much like a parts list describes which part to use for each part in the design.

Component configuration can be performed outside the architecture body which instantiates a certain component. A configuration declaration is a design unit which can be compiled separately. The particular architecture body has not to be recompiled when the binding is changed.

2.4.3 Libraries

Libraries are collections of compiled design units that are mapped to physical directories. A library is a set of components (entities and architectures) and packages.

There are two reserved library names always available to designers:

- `work`: predefined library;
- `STD`: it contains 2 packages:
  - `STANDARD` provides declaration for predefined types (real, integers, Boolean, etc);
  - `TEXTIO` contains useful subprograms that enables to perform ASCII file manipulation.

The statement `library STD;` declares STD to be a library. The statement `USE STD.STANDARD.ALL;` allows the use of all declarations in package `STANDARD` located within the library `STD`. The statement `LIBRARY ieee;` declares `ieee` to be a library. `work` is the predefined library.

3 Lesson 02-10-26

In the last lesson we introduced *combinatorial* circuits.

3.1 Datapath components

We now move on to increasingly complex digital building blocks:

- **Combinational**: gates, multiplexors, decoders, ...;
- **Sequential**: basic registers, Finite State Machines, controller;

There is a notion of *control*: *controllers* are good for systems with control inputs/outputs, as opposed to data:

- **Control input**: single bit (or a few), representing environment event or state;
- **Data input**: multiple bits representing single entity.

Our focus will be on **datapath** components at the register-transfer-level, or **RTL**. These are usually networks *controlled* by control networks, which perform functions with both:

- Components to **store** data: registers;
- Components to **transform** data: functional blocks.

3.1.1 Multiplexers

We know from theory that **multiplexers** can implement any combinational logic function (basically a variation on *Shannon's theorem*). This means that they can serve as **LUT**s (*Look Up Tables*) in FPGAs.

A few approaches can be proposed:

1. An $m \times 2^n$ to 1 multiplexer can implement m functions of n inputs. The steps are:
   - Write the truth tables for the m functions;
   - Use the n inputs as selection signals for the multiplexer;
   - Label the m multiplexer outputs with the output functions;
   - Set to 0 or 1 according to each truth table the data inputs of the multiplexer
2. An inverter and an $m \times 2^n$ to 1 multiplexer can implement m functions of n inputs. The steps are:
   - Write the truth tables for the m functions
   - Based on $n-1$ inputs identify pairs of rows in the tables
   - For each pair and output, identify an elementary function of the n-th input X (that is constant 0 or 1, or X or its negation);
   - Using the first $n-1$ inputs as indices, assign the data inputs of the multiplexer with the appropriate elementary functions.

The inverter is clearly used to generate X' (negation) in the elementary function, if needed.

3.1.2 Adders

Let's now focus on **adders**. Adders have the (apparently) trivial job of adding toghether two N bit binary numbers.

Table approach

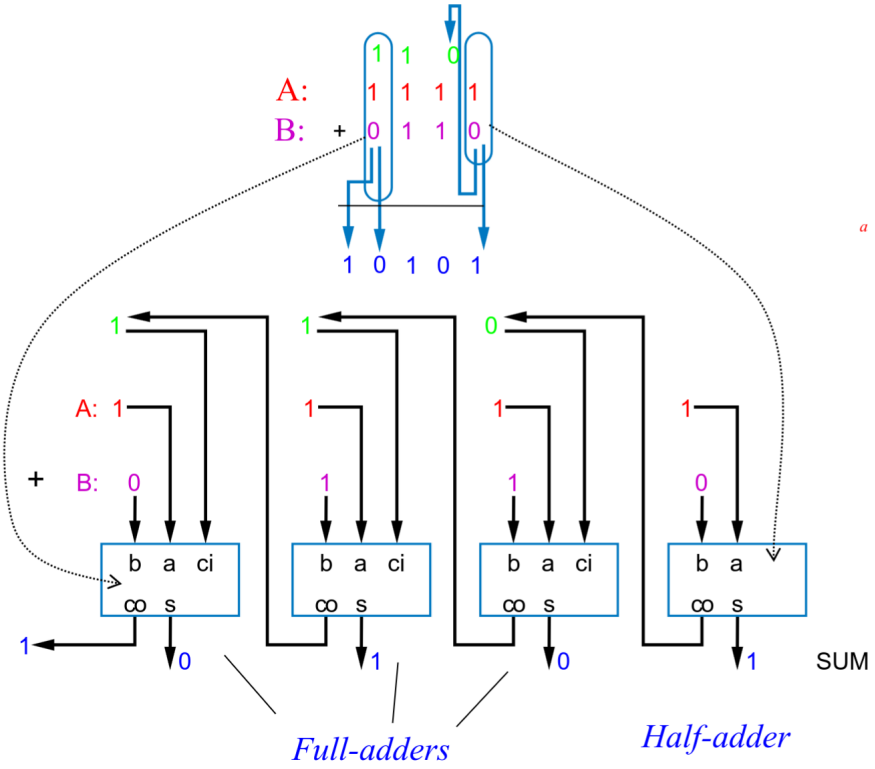
They can be designed using combinational design procedures, but this doesn't work well for typical N (8, 16, 32, ...). The approach scales terribly, as the truth table simply becomes too big to be directly expressed via Shannon expansion.

Hierarchical approach

The idea, then, is to implement in hardware what we usually do to sum numbers by hand:

1. One column at a time;
2. Compute sum of single column;
3. Carry to next column;
4. Repeat until done.

This is trivial (and boring) in hardware, with ripple carry:



Two components are used in the figure: an *half adder*, and a set of *adders*. Using half-adder and full-adders, we can build adder that adds like we would by hand:

- The component is called a *ripple-carry adder*;
- 4-bit adder shown: Adds two 4-bit numbers, generates 5-bit output;
- 5-bit output can be considered 4-bit "sum" plus 1-bit "carry out";
- We can easily build any size adder.

Half adder

The **half adder** is the most trivial component: it takes 2 bits (a and b) and returns:

- s, the LSB of the sum of the bits;
- c_{out}, the MSB of the sum of the bits (they can evaluate to binary 2, which is 10).

The half adder is easy to implement both via LUTs, but also via discrete logic gates:

$$s = a \otimes b, \quad c_{out} = a \cdot b$$

where $\otimes$ is notation for XOR.

Adder

The **adder** (or **full** *adder*) is the slightly more complicated variant of the half adder: it sums 3 inputs instead of 2. This is needed to correctly implement ripple carries, as we have the two digits at column i plus the ripple carry from column $i-1$ (which is always zero for the first digit, therefore justifying an half adder).

The full adder takes 3 bits (a, b and c_{in}) and returns:

- s, the LSB of the sum of the bits;
- c_{out}, the MSB of the sum of the bits (they can evaluate to binary 2, which is 10, or even binary 3, which is 11).

The full adder is easy to implement both via LUTs, but also via discrete logic gates or hierarchically, with half adders:

$$s = a \otimes b \otimes c_{in}, \quad c_{out} = ab + ac_{in} + bc_{in} = (a \otimes b) \cdot c_{in} + a \cdot b$$

Note that the XOR operator (again, $\otimes$) acts as a *disparity checker* operator: for $\otimes_i^n \alpha_i$, if a odd number of α_i inputs are high, the XOR operator returns true.

Note that using a full-adder instead of half-adder for first bit, we can include a "carry in" bit in the addition circuit. This is useful later when we connect smaller adders to form bigger adders.

Adder timing characteristics

Ripple carry adders have specific timing behavoir: operations have to propagate (that is, *ripple*) through all adder modules for the results to be correct. This gives non-ideal timing characteristics (more bits mean significantly more delay, with linear scaling).

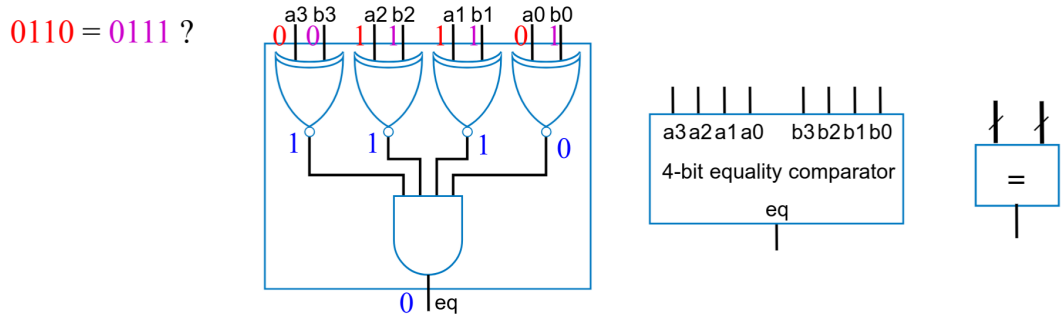
3.1.3 Comparators

We would now want to **compare** data, that is build a device which outputs 1 if two N-bit numbers are equal.

Equality comparator

If we are taking the **equality** of numbers, this is trivial. This is easy to achieve via XORs. For instance, for 4 bits:

$$eq = (a_3b_3 + a_3'b_3') \cdot (a_2b_2 + a_2'b_2') \cdot (a_1b_1 + a_1'b_1') \cdot (a_0b_0 + a_0'b_0')$$



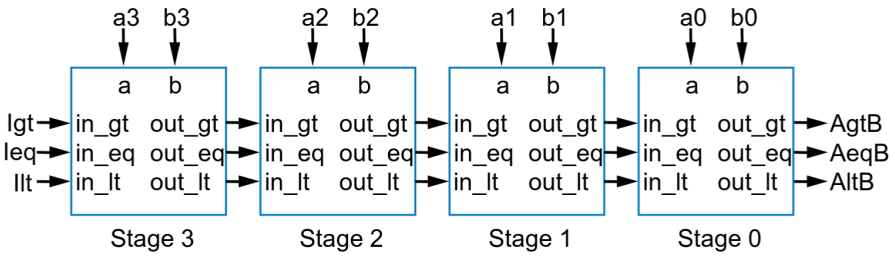
Magnitude comparator

We might also want a *magnitude*, comparator. We consider comparing by hand:

- First compare a_3 and b_3. If equal, compare a_2 and b_2. And so on;
- Stop if comparison not equal (the two bits are 0 and 1, or 1 and 0): whichever of A or B has the 1 is thus greater.

- If we never see an unequal bit pair, then A=B.

  This gives us the following circuit:



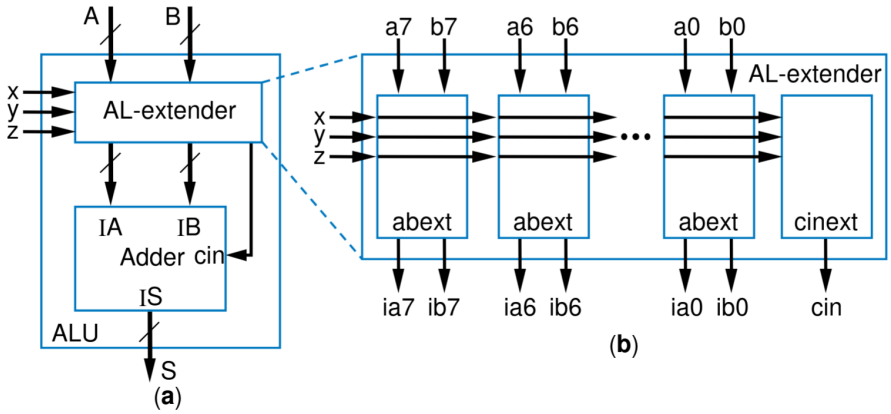
Of course, we do need a single digit comparator. Thankfully, this is easy to accomplish.

3.1.4 Arithmetic Logic Units

An **ALU** (*Arithmetic Logic Unit*) is a component that can perform various arithmetic (add, subtract, increment, etc.) and logic (AND, OR, etc.) operations, based on control inputs.

Inputs			Operation	Sample output if A=00001111, B=00000101
x	y	z		
0	0	0	S = A + B	S=00010100
0	0	1	S = A - B	S=00001010
0	1	0	S = A + 1	S=00010000
0	1	1	S = A	S=00001111
1	0	0	S = A AND B (bitwise AND)	S=00000101
1	0	1	S = A OR B (bitwise OR)	S=00001111
1	1	0	S = A XOR B (bitwise XOR)	S=00001010
1	1	1	S = NOT A (bitwise complement)	S=11110000

- The simplest way to implement an ALU is using separate components for each operation, and muxes. This, though, requires too many wires, and also wastes power computing operations when we only actually use one result at given time.
- Because of this, ALU designs don't just have separate components, multiplexed togheter. Most ALU designs use a *single adder*, plus logic in front of adder's A and B inputs. This logic is called an **arithmetic-logic extender**: the extender modifies A and B inputs so desired operation appears at output of the adder .



3.2 VHDL delay models

There are 3 ways to model delays in VHDL:

1. **Inertial delay** deletes pulses that are less than a threshold long. This is suitable for modeling delays through devise such as gate. The syntax is:
   ```
   Y1 <= not(A and B) after 7ns;
   ```
2. **Transport delay** transports all pulses, i.e., all input events are propagated to output signals. It models delays through devices with very small inertia, e.g, wires: seldom used in hardware design. The syntax is:
   ```
   Y2 <= not(A and B) transport after 7ns;
   ```
3. **Delta delay** means no propagation delays are specified. Infinitesimally small delay is automatically inserted by the simulator to preserve correct ordering of events (projected values become current values).

4 Lesson 16-10-26

4.1 Sequential logic

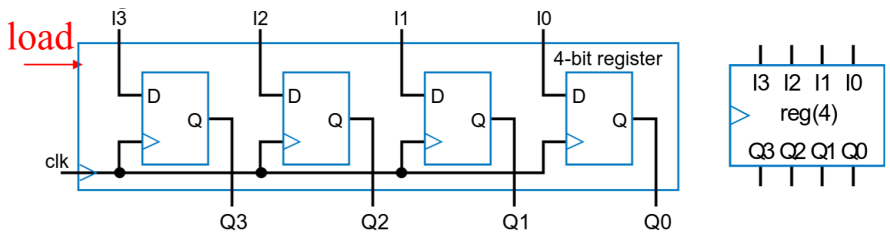
4.2 Sequential datapath components

Let's look at sequential components applied to the datapath of a circuit.

4.2.1 Registers

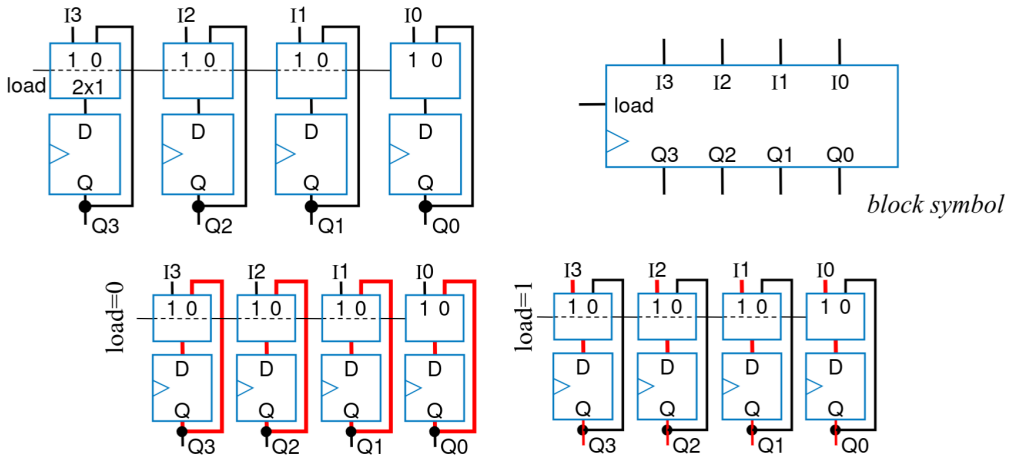
We have introduced **registers**. In general, an N-bit register stores N bits, where N is also known as the width.

A register is loaded every cycle. It is useful for implementing FSMs, as it stores encoded state (the **STAR**, *STAtus Register*).



4.2.2 Parallel load register

We can have a register *conserve* its value for more than one clock cycle by implementing a **parallel load** topology. The idea is to add a mux before the register's data input, which can select between an external (load) output and the register's own output.

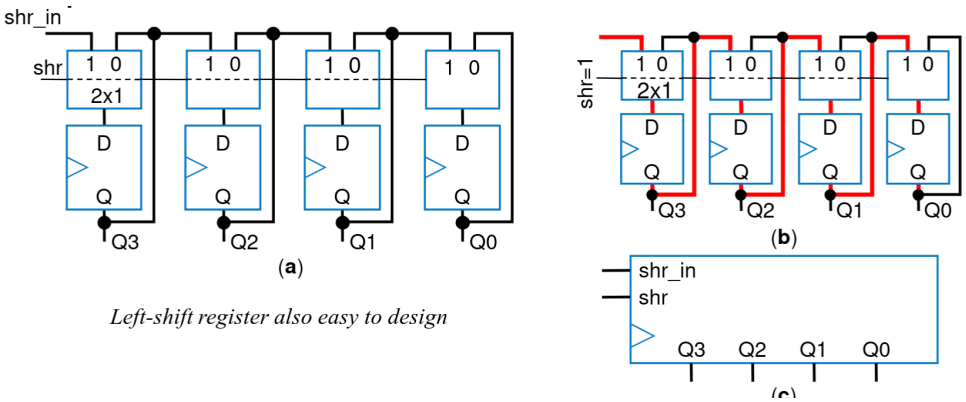


4.2.3 Shift register

An extension of the parallel load register is basically any registers which allows either to conserve or perform some operation on its contents at each clock cycle. One such popoular device is the **shift register**.

The idea of a shift register is to move each bit one position right (left) in the register width, with the rightmost (leftmost) bit is "dropped", that is 0 shifts into into leftmost (rightmost) bit.

We can easily achieve this by chaining flip-flops:



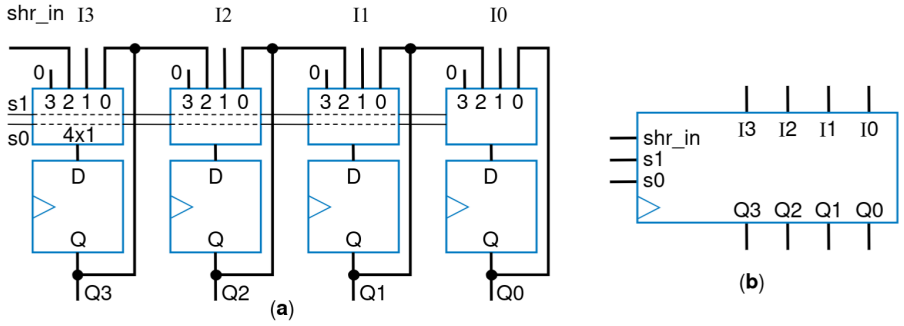
Left-shift register also easy to design

4.2.4 Multifunction register

At this point we would like to generalize registers to implement various functions, that is implement a **multifunction register**.

A multifunction register has encoded control inputs instead of decoded control inputs (one for each function): Many registers have multiple functions, such as load, shift, clear (load all 0s), and are also able to retain present value (as in a parallel load register).

These are easily designed using muxes: we ust connect each mux input to achieve the desired function.



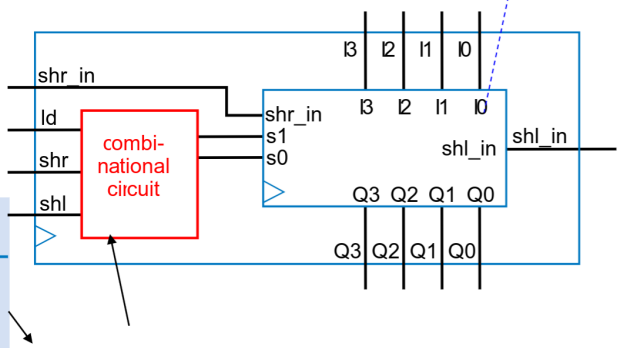
Note that usually we prefer to encode functions in binary values (with each number indexing a function). If we want single hot-encoded bits signaling a certain operation, we're better off using a combinatorial network to get back to a binary encoding:

ld	shr	shl	Operation
0	0	0	Maintain value
0	0	1	Shift left
0	1	X	Shift right
1	X	X	Parallel load

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	Shift left

Truth table for combinational circuit

Inputs ld	shr	shl	Outputs s1	s0	Note Operation
0	0	0	0	0	Maintain value
0	0	1	1	1	Shift left
0	1	0	1	0	Shift right
0	1	1	1	0	Shift right
1	0	0	0	1	Parallel load
1	0	1	0	1	Parallel load
1	1	0	0	1	Parallel load
1	1	1	0	1	Parallel load

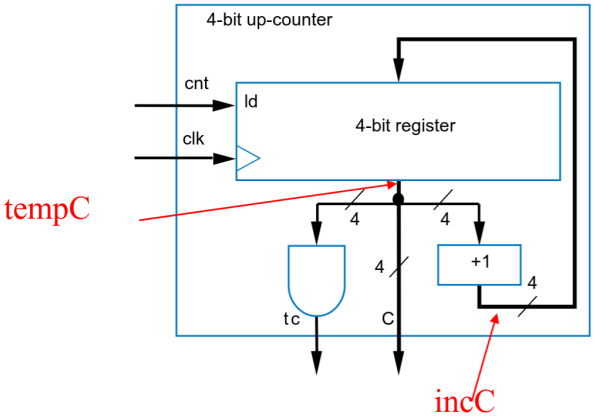


$s1 = ld'shr'shl + ld'shrshl' + ld'shrshl$
$= ld'shl + ld'shr$

$s0 = ld'shr'shl + ld$

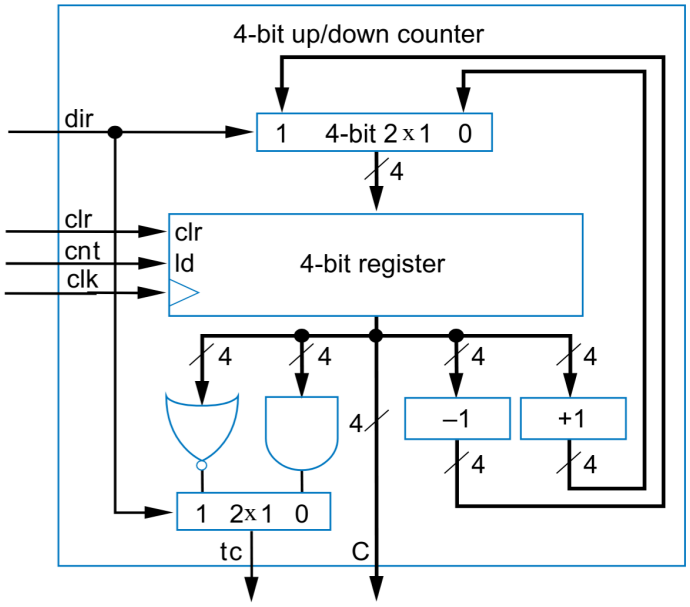
4.3 Counters and Timers

A **counter** is a sequential circuit based on an N-bit register that can increment (add 1) to its own value on each clock cycle.



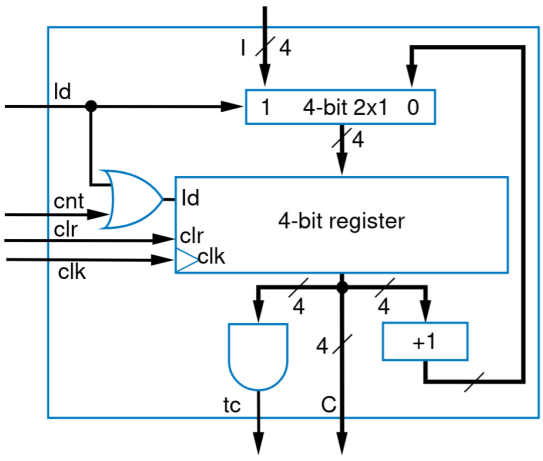
We see that the up counter design is simple: basically a multifunction register with a clear function and a increment function (extra optimizations can be hidden away in the incrementer network).

A complication we're interested in, on the other hand, is having *up/down* counters.



Let's now look into including a *load* feature in our counter.

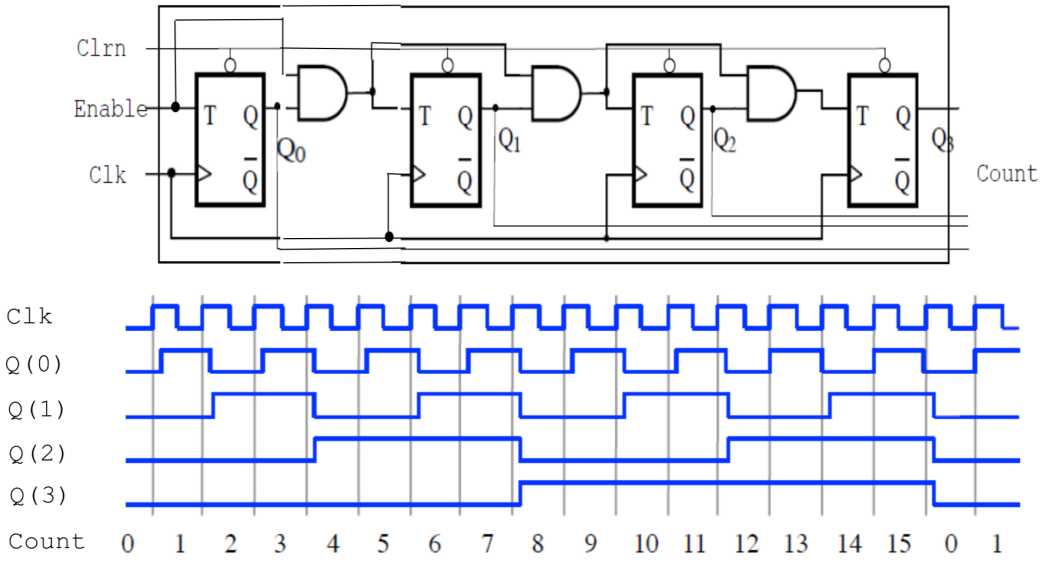
For instance, an up-counter that can be loaded with an external value is designed using a 2 to 1 mux. The `ld` input selects the incremented value or the external value. The still load the internal register when loading external value or when counting (via an OR gate).



Note that in this design `ld` has priority over `clr` (because of the conditioning combinatorial network at the inputs).

4.3.1 Synchronous implementation

We see one possible implementation for an N-bit counter with enable and clear.



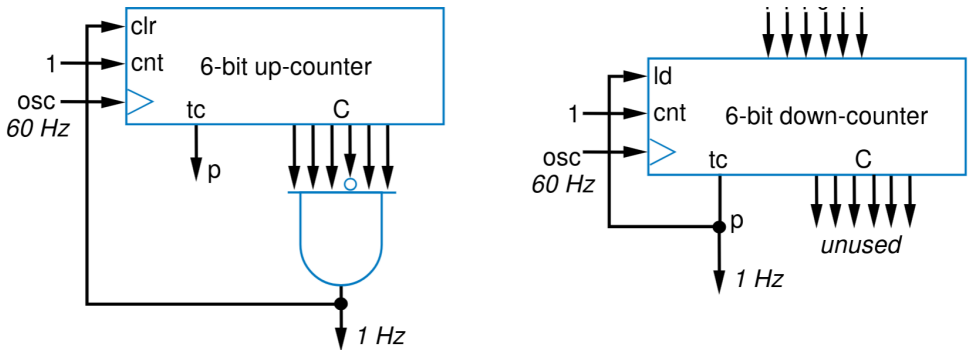
4.3.2 Modulo counters

An interesting class of counters is used to **divide clock** cycles. What we can do is basically connect the `tc` output of a down counter w/ load to its `ld` input. This means that the counter will load anything that's fed to it, the moment it stops counting (thus counting to values k, different than a power of 2).

Let's now see what we mean when we say dividing a clock. uppose have 512 Hz oscillator, but want a 2 Hz pulse. We want to divide this fast clock by 256. Noting that $2^8 = 256$, thus an 8-bit counter counts 256 cycles, we can design a divider using an 8-bit up-counter, using the `tc` output as a pulse signal.

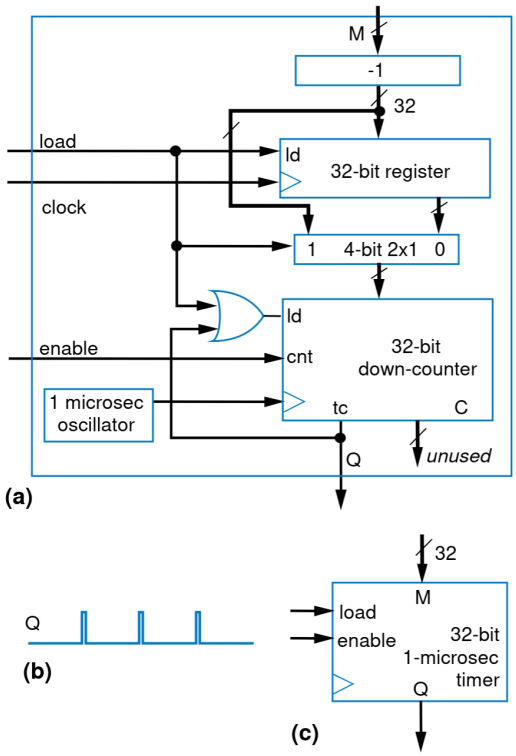
At this point the above design for a k-modulo coutner makes sense to divide signals by dividers not a power of 2. Note that an up counter approach is also possible, but more expensive, by matching with an AND gate at the output.

Let's give a final figure, showing up and down counter approaches to dividing a 60 Hz clock to a 1 Hz pulsing signal.



4.3.3 Timers

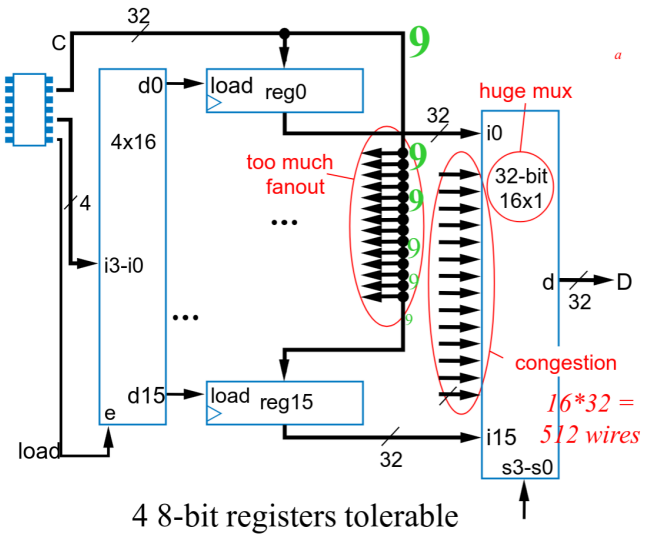
Lastly, let's see the **timer**. It is a device that pulses the output at a user-specified timer interval when enabled. The interval is clearly specified as a multiple of the base time unit. This can be designed using an oscillator, register, and down-counter:



4.4 Register files

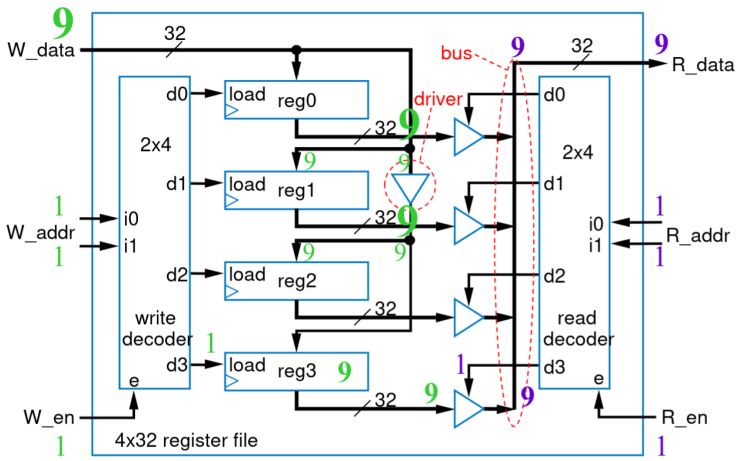
An $M \times N$ register file is a set of M registers each N bits wide. It should ideally allow for one-at-a-time, synchronous writes, and one-at-a-time (or multiple) asynchronous reads.

The simple approach, involving simple registers with load enables and a big output multiplexer, doesn't really work as it requires a huge output multiplexer (with extremely high signal degradation).



4 8-bit registers tolerable

This, a good internal design uses drivers and buses to ensure signal strength.



Internal design of 4x32 RF

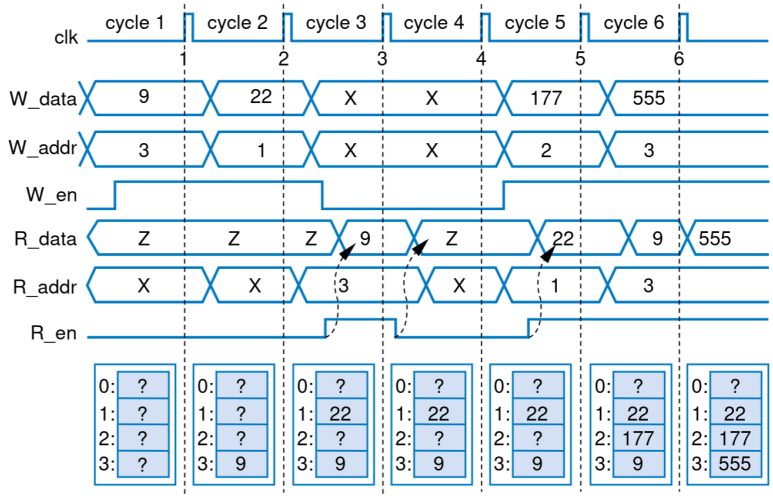
5 Lesson 23-10-26

Last lesson we looked at *register files*. Specifically, we landed on an implementation which used 2 solutions we hadn't seen before:

- **Amplifiers** or *buffers*, purely electrical devices which we used to boost the signal of the data input on the path to all registers;
- **Tri-state drivers**, which we used to select the register that drives the output data bus (leaving the others in high impedance).

5.0.1 Register file timing

Let's quickly see a graph of timing constants for register files. We see that operations are clock-synchronized, with data inputs and outputs (and corresponding address lines) having to be stable at clock rising edges.



5.1 RTL design

So far we focused on behavioral and maybe dataflow designs of combinatorial and sequential devices. Now we want to move on to **RTL** (*Register Transfer Level*) designs, which really sit at an higher level of abstraction.

We already defined embedded systems as digital electronic systems focused not on general purpose but on specific applications.

5.1.1 Metrics

Among the design metrics we are optimizing for there's an obvious design goal:

- Construct an implementation with desired functionality.

The key design challenge is to simultaneously optimize numerous *design metrics*. A **design metric** is a measurable feature of a system's implementation.

Some common metrics are:

- **Unit cost**: the monetary cost of manufacturing each copy of the system, excluding NRE cost;
- **NRE cost** (*Non-Recurring Engineering cost*): the one-time monetary cost of designing the system;
- **Size**: the physical space required by the system;
- **Performance**: the execution time or throughput of the system;
- **Power**: the amount of power consumed by the system;
- **Flexibility**: the ability to change the functionality of the system without incurring heavy NRE costs;
- **Time-to-prototype**: the time needed to build a working version of the system;
- **Time-to-market**: the time required to develop a system to the point that it can be released and sold to customers;
- **Maintainability**: the ability to modify the system after its initial release;
- Other metrics include *correctness*, *safety*, ...

5.1.2 Technologies

Optimizing means choosing techonologies, so let's go over the 3 main types:

- **VLSI** (*Very Large Scale Integration*), also known as full custom, all layers are optimized for an embedded system's particular digital implementation. This means placing and sizing transistors, routing wires, ...

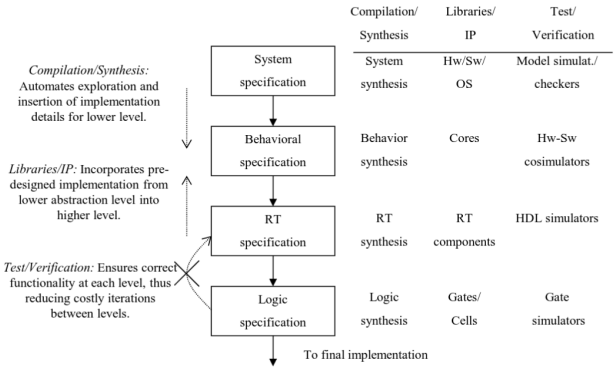
  The benefits are excellent performance, small size, low power, at the cost of high NRE cost and time-to-market;
- **ASIC**s (*Application Specific Integrated Circuits*), lower layers are fully or partially built, and designers are left with routing of wires and maybe placing some blocks.

  The benefits are good performance, good size, less NRE cost than a full-custom solution. Time-to-market and costs are still pretty high, though;
- **PLDs** (*Programmable Logic Devices*): all layers already exist, designers can purchase an IC, and the connections on the IC are either created or destroyed to implement desired functionality (usually via a software programmer). *Field-Programmable Gate Array* (**FPGA**s) are very popular PLDs.

The benefits are low NRE costs, almost instant IC availability, while the drawback is that PLDs are bigger, expensive (per-unit) and, power hungry. hungry, slower

5.1.3 Design flow

What we are interested in in this course is the design technology, that is the manner in which we convert our concept of desired system functionality into an implementation, rather than technology in it of itself. We know, for instance, that our tools allow us to easily convert our designs to FPGA programmer instructions.



5.1.4 Recap

In the previous lessons we saw:

- **Combinatorial** networks, so ways to map equations and truth tables to logic;
- **Sequential** networks, so FSMs, registers, etc...
- **Datapath** components based on combinatorial and sequential designs which allowed simple actual elaboration on data.

We know get to the point of **RTL** design, where we intend to:

- Capture circuit *behavior* via high level FSMs;
- Convert these FSMs to circuits via the **controller** + **datapath** design model (where a controller controls a datapath forming a **processor**).

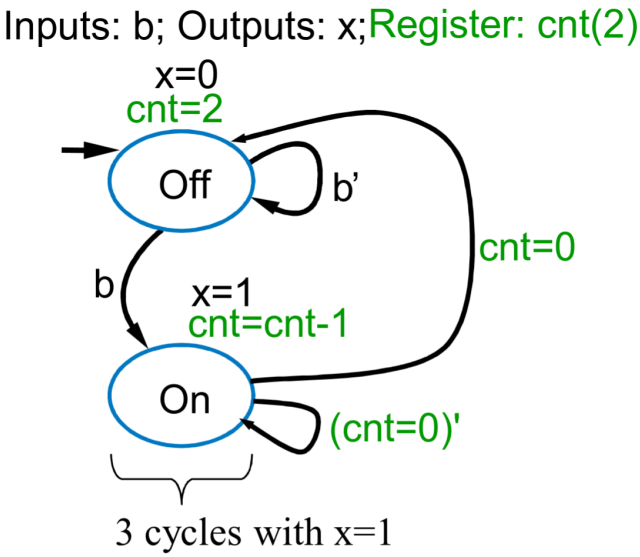
5.2 High-Level State Machines

Some behaviors are too complex for equations, truth tables, or simple FSMs. This is where we extend FSMs with features such as *state* variables, forming **higher-level** SMs.

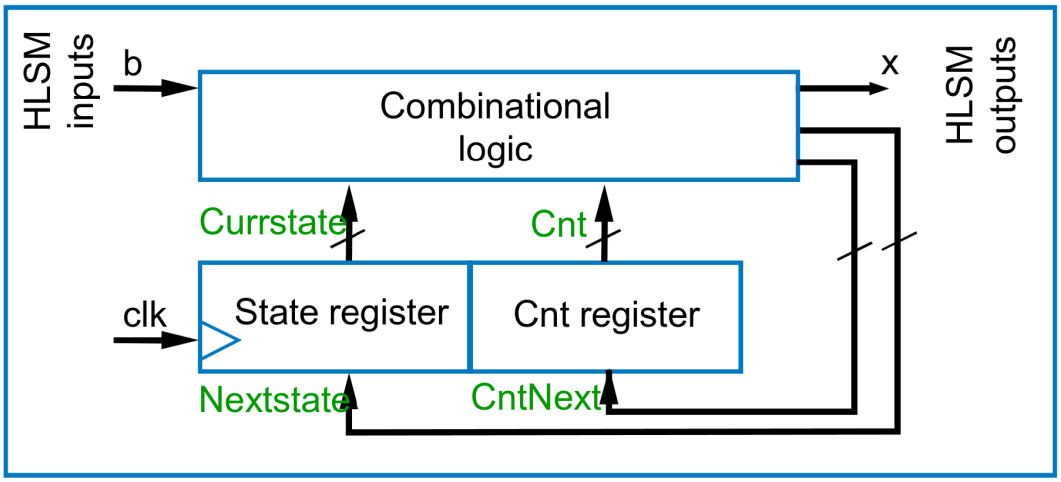
More specifically, the **HLSM** (*High-Level State Machine*) model extends FSM with:

- Multi-bit input/output;
- Local storage (state variables);
- Arithmetic operations on those state variables.

Let's take a simple example of a variable counter. For an N-counter, we could design an $N + 1$ state FSM, but that becomes cumbersome fast. A better approach is to use an FSM governing a register.



Note that this can be interpreted as an FSM where the status register is *"extended"* by the bits of the counter register.



6 Lesson 30-10-26

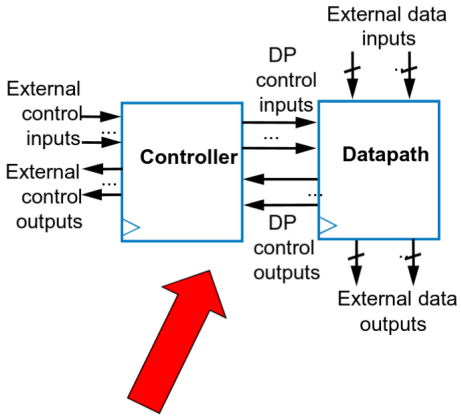
6.1 RTL design process

We know get to actual **RTL** (*Register Transfer Level*) design.

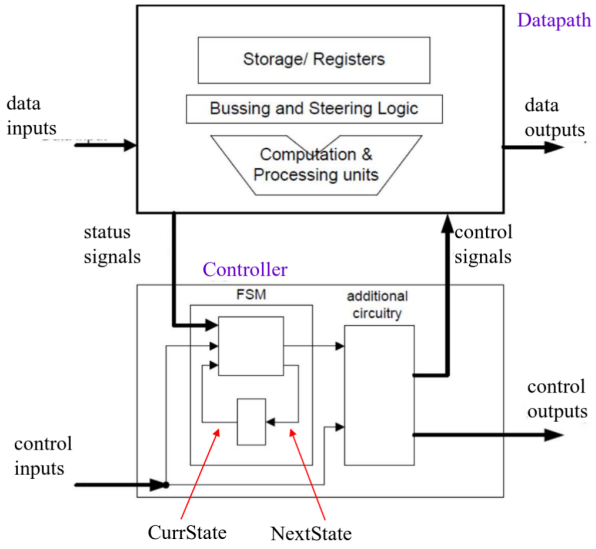
The idea is to:

- Capture behavior (via a behavioral**HLSM**);
- Convert the behavior to a circuit: we need a target architecture, that is a *datapath* capable of the HLSM's data operations, and a *controller* able to control the datapath.

  The model that works for us is the **FSM-D** model: *FSM with Datapath*.
- The **controller** is a classical FSM, easy to design with our known behavioral models, easy to synthesize;
- The **Datapath**, given in a structural or behavioral description, contains the functions controlled by the datapath.



Note that the datapath is not necessarily only combinatorial: it can contain sequential components (as we saw earlier with *Sequential Datapath Compoenents*).



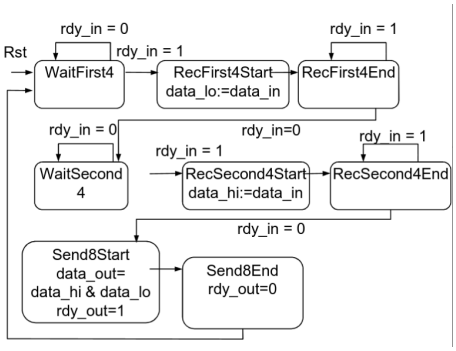
Let's now look at the actual design procedure.

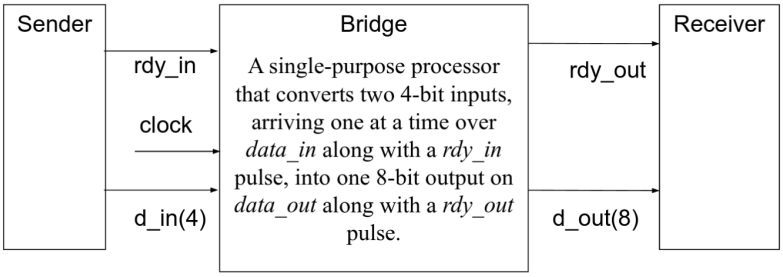
1. Describe the system's desired behavior as a high-level state machine. The state machine consists of states and transitions. The state machine is "high-level" because the transition conditions and the state actions are more than just Boolean operations on single-bit inputs and outputs;
2. Create a datapath to carry out the data operations of the high-level state machine;
3. Connect the datapath to a controller block. Connect external control inputs and outputs to the controller block;
4. Convert the high-level state machine to a finite-state machine (FSM) tor the controller, by replacing data operations with setting and reading of control signals to and from the datapath.

We now introduce two categories of FSM-D design.

- In **control dominated design** we have complex controller controlling a simple datapath.

Let's take, as an example, a *bridge* component. This is a single-purpose processor that converts two 4-bit inputs, arriving one at a time over `data_in` along with a `rdy_in` pulse, into one 8-bit output on `data_out` along with a `rdy_out` pulse.
Explained via the state diagram:






```
ENTITY Bridge IS
PORT (Clk:     IN std_logic;
      Rst:     IN std_logic;
      rdy_in:  IN std_logic;
      d_in:    IN std_vector(3 DOWNTO 0);
      rdy_out: OUT std_logic;
      d_out:   OUT std_vector(7 DOWNTO 0));
END Bridge;
```

VHDL: Embedded Systems RTL Design

- The alternative is **data dominated design**, where we have an extensive datapath and a simple controller. A good example is a filter, which converts digital input stream to new digital output stream, for instance to remove *noise*.



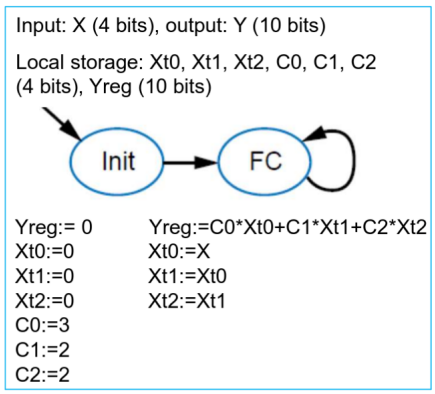
A simple filter gives as an output the weighted sum of the last N values. Small N means less filtering, and large N means more filtering, but less sharp output.
If the weights of the sum can be chosen, then this is a **FIR** (*Finite Impulse Response*) filter. Simply a configurable weighted sum of past input values:

$$Y(t) = W_0 \cdot X(t) + W_1 \cdot X(t-1) + C_w \cdot X(t-2)$$

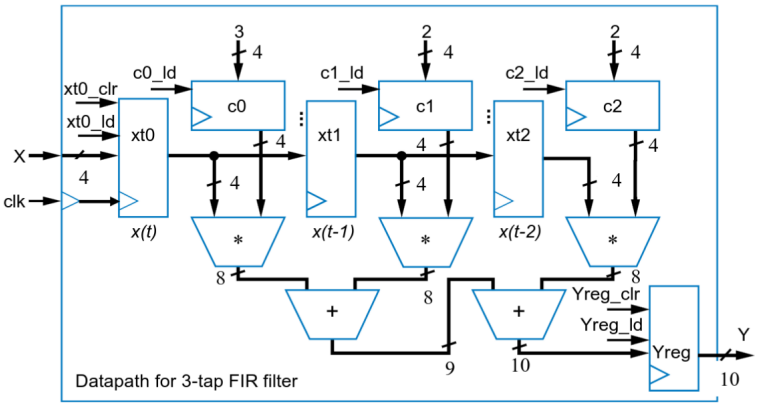
This is known as a *3-tap* filter, as it involves 3 multiplicatory terms. Tens of taps per filter are more common. Know that this is a very general filter: the designer sets the constants (W_0, W_1, W_2) to define a specific filter behavior.

From the hardware standpoint, this is simply a set of multiplication circuits, plus an adder for all terms.

The FSM is trivial, as mostly we want to keep state in our storage devices (for each sampled tap value).



Now, the datapath for this is far more complex, because the operations we seek to implement is arithmetically more complex:



6.1.1 Critical paths

Designers of digital circuits often want the fastest possible performance. This means high clock frequency.

Frequency is limited by the longest register-to-register delay in our datapath, known as the **critical path**. If clock is any faster, incorrect data may be stored into our registers.

6.2 Algorithmic-Level Behavior

Good news! An abstraction level higher than RTL.

HLSM was already pretty high-level, but even higher is sometimes needed for certain applications. In an **algorithmic-level** description, behavior is described as sequence of steps, similar to sequential program.

Video compression, for example, computes sum of absolute differences (SAD) of corresponding pairs in two arrays. It is a common task in video compression to determine difference between two successive video frames.

Other examples are the Greatest Common Divisor and then Serial-parallel multiplier.

6.2.1 Sum of Absolute Differences

In video compression we might need to quickly determine whether two frames are similar enough to just send difference for second frame.

A technique we can use is **SAD** (*Sum of Absolute Differences*).

As an example, we compare corresponding 16x16 "blocks", treating each 16x16 block as a 256-byte array. From these we compute the absolute value of the difference of each array item, sum those differences:if above a threshold, send complete frame for second frame; if below, can use difference method (using another technique, not described here).

This is most easily described via an algorithm. For instance, in VHDL:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.std_logic_signed.ALL;

ENTITY SAD IS
  PORT (go: IN std_logic;
  SAD_out: OUT integer;
  Clk, Rst: IN std_logic );
END SAD;

ARCHITECTURE SAD_Alg OF SAD IS
  TYPE SADArrayType IS ARRAY(0 TO 255)
  OF std_logic_vector(7 DOWNTO 0);
  -- somehow declare A and B here
BEGIN
  PROCESS
  VARIABLE Sum: integer;
  BEGIN
    IF (NOT(go='1')) THEN
      WAIT UNTIL go='1';
    END IF;

    Sum := 0;
    FOR I IN 0 TO 255 LOOP
      Sum := Sum + Conv_Integer(ABS(A(I) - B(I)));
    END LOOP;

    WAIT FOR 50 ns; -- model comb. delay
    SAD_out <= Sum;
  END PROCESS;
END SAD_Alg;
```